

NIRVANA HOMES

Case Study Analysis

Property price modelling and sales-speed prediction

Python · pandas · scikit-learn

Dataset	1,400 property listings
Targets	listed_price_lakhs (regression); sold_within_45_days (classification)
Models	Linear Regression, Decision Tree, Random Forest
Split	80% train / 20% test, random_state = 42

1. Objective and Approach

Nirvana Homes wants to understand what drives the listed price of a property and which listings are likely to sell slowly. This analysis answers four questions using one dataset of 1,400 listings across Bengaluru, Hyderabad and Pune:

- **Q1 Pricing model:** fit a multiple linear regression for *listed_price_lakhs* and identify the strongest price drivers.
- **Q2 Error analysis:** find where the pricing model is least reliable.
- **Q3 Sales speed:** classify whether a listing sells within 45 days using a decision tree, compared with a majority-class baseline.
- **Q4 Alternative model:** test whether a random forest does better, and check how much listing photo quality matters.

The same 17 predictors are used for both tasks: city, locality tier, BHK, area, building age, floor number, total floors, distance to metro, amenity score, furnishing, parking spots, corner unit, monthly maintenance, broker experience, number of photos, agent photo-quality score and festival-season flag.

2. Data Loading and Audit

The CSV is loaded with pandas and audited for shape, data types, missing values and exact duplicate rows. Duplicates, if any, are dropped before modelling.

```
df = pd.read_csv(file_path)
print(df.shape); print(df.dtypes); print(df.isnull().sum())
num_duplicates = df.duplicated().sum()
if num_duplicates > 0:
    df = df.drop_duplicates()
```

Both targets are then inspected.

Target	Summary
listed_price_lakhs	n = 1,400; mean 75.44; std 41.34; min 14.36; 25% 47.73; median 65.55; 75% 92.90; max 376.84
sold_within_45_days	0 (slow): 790 listings (56.4%) 1 (sold within 45 days): 610 listings (43.6%)

Price is right-skewed (mean above median, maximum about 5.7 times the median), so a few expensive listings will have large influence on the regression. The classes are only mildly imbalanced, with slow listings forming the majority.

Missing values: *amenity_score* had missing entries, which are filled with the median value of 4.8. Categorical variables are one-hot encoded with *drop_first=True*, so the reference categories are Bengaluru, locality tier 1, Fully-Furnished and *festival_season_listing* = 0.

3. Q1: Linear Regression Pricing Model

```
X_reg = pd.get_dummies(X_reg_raw, columns=categorical_cols, drop_first=True, dtype=float)
X_train, X_test, y_train, y_test = train_test_split(X_reg, y_reg, test_size=0.20, random_state=42)
reg_model = LinearRegression().fit(X_train, y_train)
```

Model performance

Metric	Training	Testing
R ²	0.8937	0.9034
RMSE (lakhs)	13.56	12.50
MAE (lakhs)	8.66	8.69

The model explains about 90% of the variation in listed price on unseen data. Training and test scores are almost identical, so the model is not overfitting. The typical error (MAE 8.69 lakhs) is about 11.5% of the mean price of 75.4 lakhs.

Coefficients (reference: Bengaluru, tier 1, Fully-Furnished; intercept 39.73)

Feature	Coefficient	Feature	Coefficient
locality_tier_3	-43.70	area_sqft	+0.0718
locality_tier_2	-27.57	amenity_score	+1.26
city_Hyderabad	-16.55	parking_spots	+2.56
city_Pune	-11.46	is_corner_unit	+3.80
bhk	-6.08	age_years	-0.70
furnishing_Unfurnished	-4.65	distance_to_metro_km	-1.52
furnishing_Semi-Furnished	-4.16	festival_season_listing_1	-0.53

Raw coefficients are measured in different units (square feet, rupees, years), so they cannot be ranked directly. Each coefficient is therefore multiplied by its feature standard deviation to give the price change for a one-standard-deviation move.

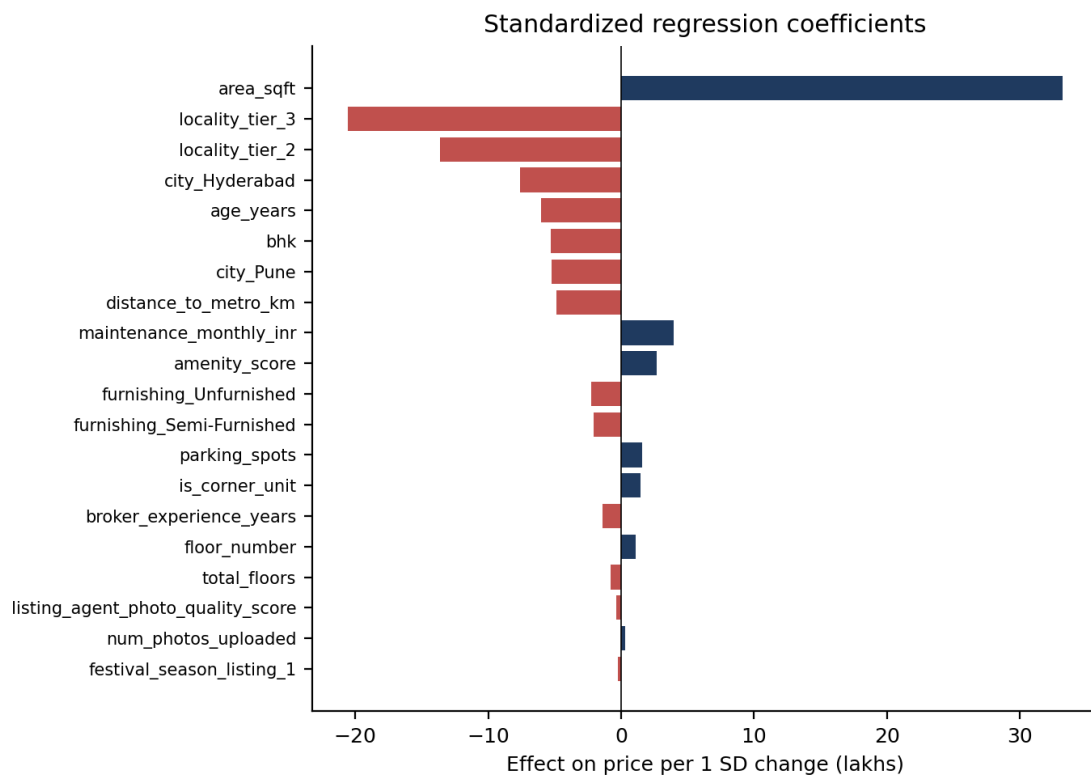


Figure 1. Standardized coefficients: effect on price per one standard deviation (lakhs).

Key findings

- **Size dominates:** area_sqft has by far the largest effect (+33.2 lakhs per standard deviation).
- **Location is next:** moving from tier 1 to tier 3 lowers price by about 43.7 lakhs, and to tier 2 by about 27.6 lakhs. Hyderabad and Pune price below Bengaluru by about 16.6 and 11.5 lakhs.
- **Age and metro distance reduce price:** about 0.7 lakhs per year of age and 1.5 lakhs per km from metro.
- **Listing-presentation variables barely matter for price:** photo count, photo-quality score and festival season all have effects below 0.35 lakhs per standard deviation.
- **Note on bhk:** its negative coefficient (-6.08) appears because area is already in the model. For the same area, more bedrooms means smaller rooms, which is valued lower. It does not mean adding a bedroom lowers price.

4. Q2: Where Does the Pricing Model Go Wrong?

For every test listing, the error is calculated as actual price minus predicted price. The ten largest absolute errors are listed below.

Listing	City	Tier	BH K	Area	Age	Actual	Predicted	Abs. error
NH-00405	Bengaluru	1	4	2191	7	276.11	186.07	90.04
NH-00608	Hyderabad	3	4	2471	12	78.10	129.41	51.31
NH-00704	Bengaluru	2	4	2270	9	212.24	162.87	49.37
NH-00554	Pune	3	3	1686	22	50.08	93.83	43.75
NH-01296	Bengaluru	1	3	1909	11	208.47	166.95	41.52
NH-00511	Bengaluru	1	4	2137	8	225.90	185.20	40.70
NH-01188	Bengaluru	3	4	1941	27	73.51	100.76	27.25
NH-00697	Bengaluru	2	4	2165	7	179.85	154.69	25.16
NH-01333	Pune	1	4	2467	13	209.49	185.40	24.09
NH-00175	Pune	2	4	2434	3	196.43	173.38	23.05

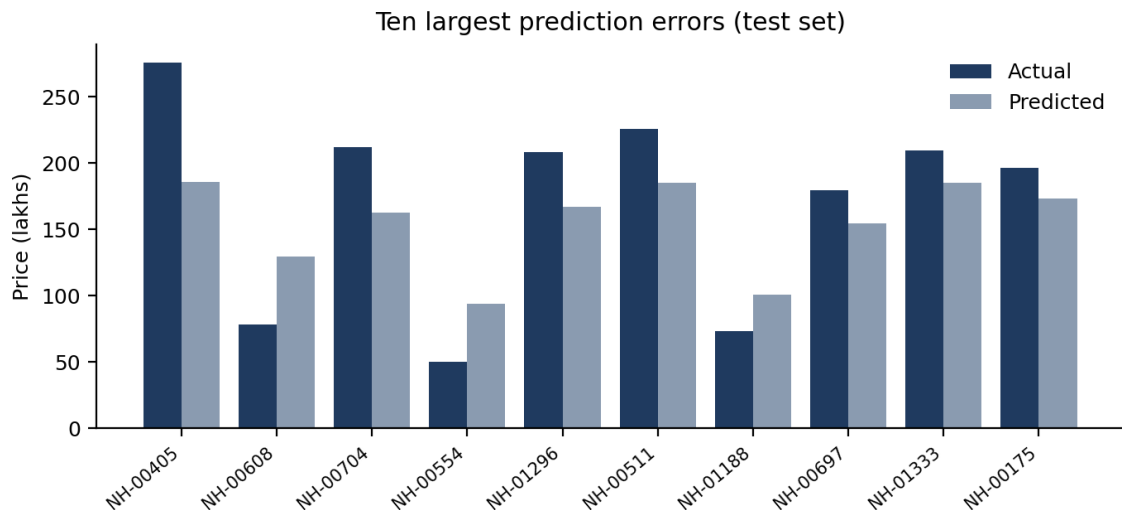


Figure 2. Actual vs predicted price for the ten worst predictions.

Eight of the ten worst errors are 4 BHK flats, and all ten are large flats (area above the 1,507 sq ft upper quartile). The biggest miss is a Tier 1 Bengaluru 4 BHK listed at 276 lakhs but predicted at 186 lakhs.

Segment analysis. A large flat is defined as area at or above the upper quartile (1,507 sq ft), and an old building as age at or above the upper quartile (20 years).

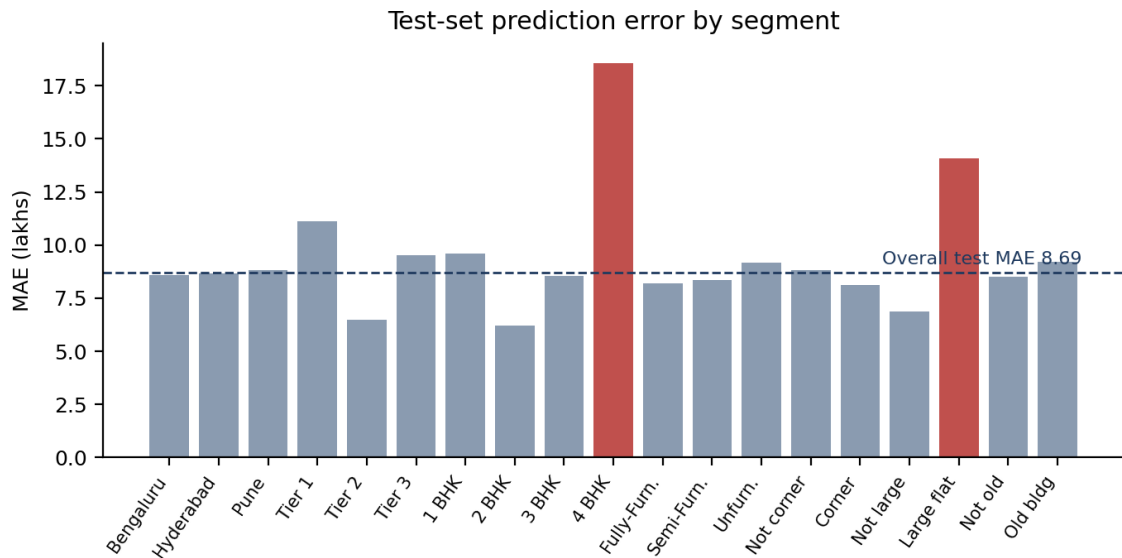


Figure 3. Mean absolute error by segment (red = clearly higher than average).

Segment	N	MAE	RMSE	Mean error
Locality tier 1	67	11.11	16.73	-1.36
Locality tier 2	111	6.46	9.10	+1.58
Locality tier 3	102	9.52	12.45	-2.40
BHK 2	108	6.21	7.77	-0.79
BHK 4	25	18.58	27.64	+6.05
Large flat: No	210	6.89	8.54	-0.19
Large flat: Yes	70	14.08	20.16	-1.73
Old building: Yes	69	9.23	11.66	-1.91
Corner unit: Yes	50	8.12	15.15	+1.53

Findings

- City, furnishing, corner unit and building age make little difference to accuracy. Average errors by city sit between 8.6 and 8.8 lakhs.
- **The model is weakest on 4 BHK flats** (MAE 18.6, about three times that of 2 BHK) **and on large flats** (MAE 14.1, about twice small flats). Price spread grows with size, which a straight-line model cannot fully capture.
- Tier 1 localities also have higher errors (MAE 11.1) than tier 2 (6.5).
- **Suggested improvement:** model log(price) or add interactions between area and locality tier or city.

5. Q3: Sales-Speed Classification (Decision Tree)

The target is *sold_within_45_days*. Because the business concern is listings that will sell slowly, **class 0 (slow) is treated as the class of interest**, and precision, recall and F1 are reported for both classes. A decision tree limited to depth 5 keeps it interpretable. A majority-class dummy model gives the baseline.

```
dt_model = DecisionTreeClassifier(max_depth=5, random_state=42).fit(X_train_cls, y_train_cls)
dummy_model = DummyClassifier(strategy='most_frequent').fit(X_train_cls, y_train_cls)
```

The test set has 280 listings: 159 slow and 121 fast.

Metric	Majority baseline	Decision tree
Accuracy	0.568	0.682
Precision (slow)	0.568	0.688
Recall (slow)	1.000	0.805
F1 (slow)	0.724	0.742
Precision (fast)	0.000	0.670
Recall (fast)	0.000	0.521
F1 (fast)	0.000	0.586

Reading the results

- The tree beats the baseline by about 11 percentage points of accuracy (68.2% vs 56.8%).
- It correctly flags 128 of 159 slow listings (recall 80.5%) and is right 68.8% of the time when it flags one.
- The baseline reaches 100% slow recall only by labelling every listing slow, so it finds no fast sellers at all. The tree finds 63 of 121 (52%).
- It wrongly labels 58 fast-selling listings as slow, so the signal is moderate rather than strong.

6. Q4: Random Forest Alternative

```
rf_model = RandomForestClassifier(n_estimators=100, random_state=42).fit(X_train_cls, y_train_cls)
```

Metric	Decision tree	Random forest
Accuracy	0.682	0.643
Precision (slow)	0.688	0.661
Recall (slow)	0.805	0.761
F1 (slow)	0.742	0.708
Precision (fast)	0.670	0.608
Recall (fast)	0.521	0.488
F1 (fast)	0.586	0.541

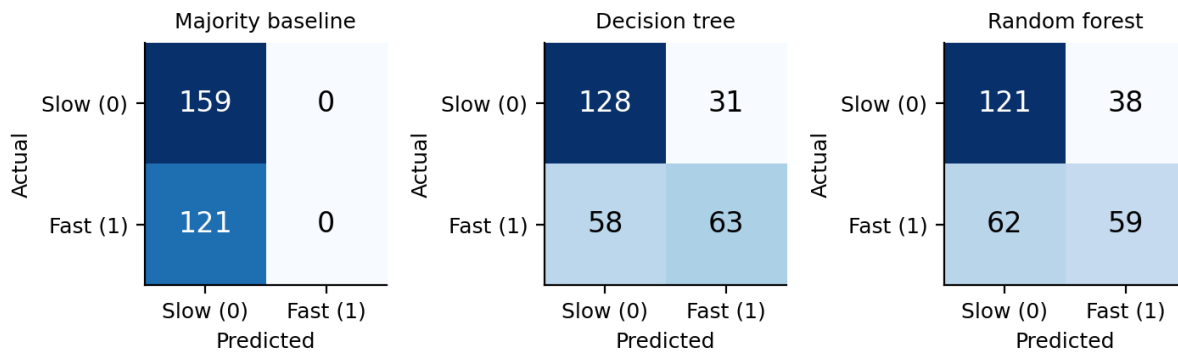


Figure 4. Confusion matrices on the 280-listing test set.

On this split, the random forest does **not** beat the single decision tree (accuracy 64.3% vs 68.2%). With only 280 test rows, a gap of four points is within normal sampling noise (roughly ± 5.5 points), so the fair reading is that the two models perform similarly.

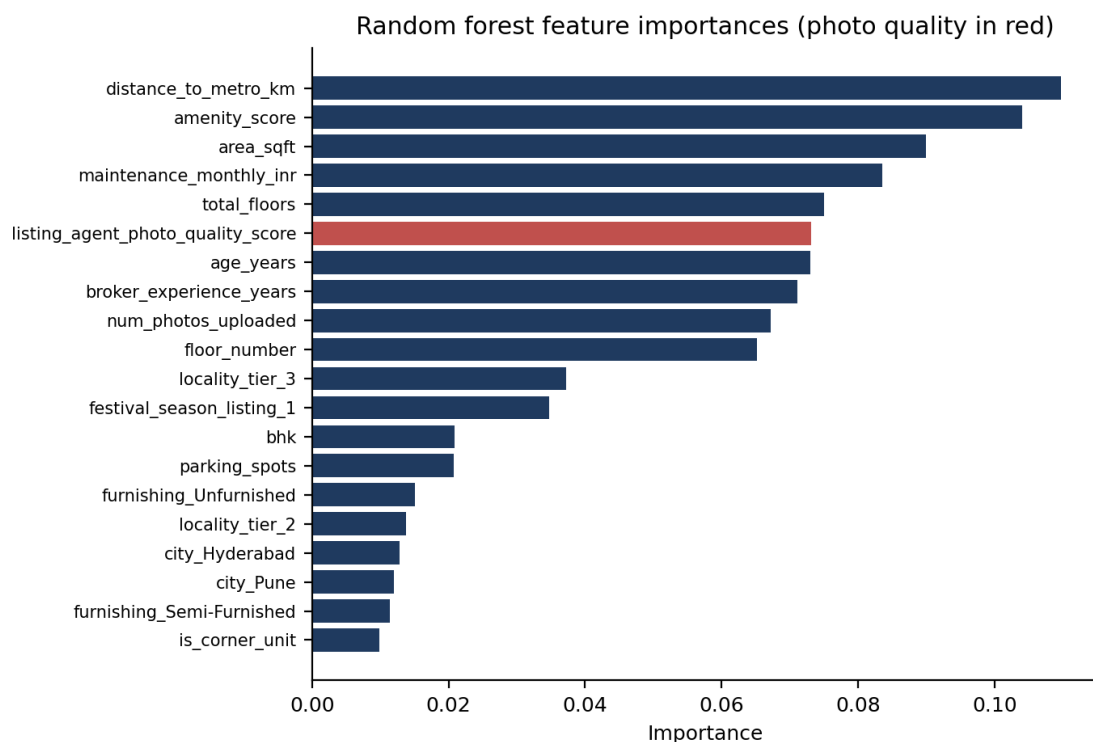


Figure 5. Random forest feature importances.

Importance results. Distance to metro (0.110) and amenity score (0.104) rank first and second, followed by area and maintenance. **listing_agent_photo_quality_score ranks 6th (0.073).** That is close to features ranked 7 to 10, so it is a moderate contributor, not a dominant lever. Note also that its effect on price in Q1 was negligible.

7. Summary of Findings

Question	Result
Q1 Pricing model	R ² 0.903 on test data; MAE 8.69 lakhs (about 11.5% of mean price). Area, locality tier and city are the main price drivers.
Q2 Error analysis	Errors are concentrated in 4 BHK and large flats (MAE 18.6 and 14.1 lakhs vs about 6 to 7 for small flats).
Q3 Decision tree	68.2% accuracy vs 56.8% baseline; catches 80.5% of slow listings.
Q4 Random forest	64.3% accuracy; no clear improvement over the tree. Photo quality ranks 6th of 20 features, a moderate contributor.

Business implications

- Use the regression model as a pricing guide for typical 1 to 3 BHK listings, but get manual review for 4 BHK and large premium flats.
- Locality, size and metro access matter most for both price and sale speed. Photo count and photo quality have little influence on price.
- Use the sales-speed model as an early-warning screen rather than a firm decision tool, since it still mislabels 58 fast sellers as slow.

Limitations and next steps

- Results come from a single 80/20 split. Cross-validation would give more reliable comparisons between models.
- Tree depth and random forest settings were not tuned.
- Price is skewed; a log-price model or interaction terms may improve accuracy on large flats.
- Permutation importance would give a more dependable ranking of features than the built-in importance scores.